

78. 子集

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Medium	(Not Available)	(Not Available)	-	-	0

- **Tags:** 位运算, 数组, 回溯
- **Companies:** (Not Available)

给你一个整数数组 `nums`，数组中的元素 互不相同。返回该数组所有可能的子集（幂集）。

解集 不能包含重复的子集。你可以按 任意顺序返回解集。

示例 1:

输入: `nums = [1,2,3]`

输出: `[[], [1], [2], [1,2], [3], [1,3], [2,3], [1,2,3]]`

示例 2:

输入: `nums = [0]`

输出: `[[], [0]]`

提示:

- `1 <= nums.length <= 10`
- `-10 <= nums[i] <= 10`
- `nums` 中的所有元素 互不相同

Discussion | Solution

解决方法

1. 问题理解

目标: 找到给定数组（元素互不相同）的所有可能子集（包括空集）。输入: 一个整数数组 `nums`。输出: 一个包含 `nums` 所有子集的列表。子集是原数组中零个或多个元素的组合。

幂集: 一个集合的所有子集构成的集合。对于包含 `n` 个元素的集合，其幂集包含 2^n 个子集。

2. 思路分析

这个问题同样可以用回溯算法解决。与全排列不同，子集问题关注的是元素是否被“选择”加入子集，而不关心元素的顺序。

方法一：回溯法（选择/不选择） 对于 `nums` 中的每一个元素，我们都有两种选择：要么将它包含在当前子集中，要么不包含。

1. 定义回溯函数 `backtrack(index, current_subset)`:
 - `index`: 当前正在考虑 `nums` 数组中的哪个元素的索引。
 - `current_subset`: 当前构建的部分子集。
2. 递归终止条件:
 - 当 `index` 到达数组末尾 (`index == len(nums)`) 时，表示我们已经对所有元素做出了选择（包含或不包含）。
 - 此时，`current_subset` 就是一个完整的子集。将其副本加入结果列表 `result`。
 - 返回。
3. 递归步骤（两种选择）:
 - 选择不包含 `nums[index]`: 直接递归到下一个元素 `backtrack(index + 1, current_subset)`。
 - 选择包含 `nums[index]`:
 - 将 `nums[index]` 加入 `current_subset`。
 - 递归到下一个元素 `backtrack(index + 1, current_subset)`。
 - 撤销选择（回溯）: 将 `nums[index]` 从 `current_subset` 中移除，以保证不影响“不包含”分支的探索。

这种方法直观地模拟了每个元素的两种选择，但实现起来需要注意状态的恢复。

方法二：回溯法（组合视角） 另一种更常用的回溯思路是，将生成子集看作是从数组中选择 0 个、1 个、2 个…直到 `n` 个元素的过程。

1. 定义回溯函数 `backtrack(start_index, current_subset)`:
 - `start_index`: 本轮选择的起始位置。为了避免产生重复的组合（如 `[1, 2]` 和 `[2, 1]` 被视为不同排列，但在子集中是相同的），我们规定每次选择只能从 `start_index` 及之后的元素中选取。
 - `current_subset`: 当前构建的部分子集。
2. 收集结果:
 - 在递归函数的入口处，就将当前 `current_subset` 的副本加入结果列表 `result`。因为从根节点到任意节点的路径本身就构成一个子集（包括空集）。
3. 遍历选择列表:
 - 从 `start_index` 开始，遍历到数组末尾 (`len(nums) - 1`)。设当前遍历到的索引为 `i`。
 - 做选择: 将 `nums[i]` 加入 `current_subset`。
 - 递归进入下一层决策: 调用 `backtrack(i + 1, current_subset)`。注意，下一个选择的起始位置是 `i + 1`，确保不会选择 `i` 之前的元素，避免重复组合。
 - 撤销选择（回溯）: 将 `nums[i]` 从 `current_subset` 中移除。
4. 主函数调用:
 - 初始化结果列表 `result = []`。
 - 调用 `backtrack(0, [])` 开始回溯。
 - 返回 `result`。

这种方法更简洁，也更符合组合问题的回溯范式。

方法三：迭代法（基于位运算） 如果数组长度为 n ，那么总共有 2^n 个子集。每个子集可以对应一个 n 位的二进制数，其中第 i 位为 1 表示包含 `nums[i]`，为 0 表示不包含。

1. 遍历从 0 到 $2^n - 1$ 的所有整数 i 。
2. 对于每个整数 i ，将其看作一个 n 位二进制数。
3. 遍历二进制数的每一位（从 0 到 $n-1$ ）。设当前位为 j 。
4. 如果 i 的第 j 位为 1（可以通过位运算 $(i \gg j) \& 1 == 1$ 判断），则将 `nums[j]` 加入当前子集。
5. 构建完当前子集后，将其加入结果列表。

这种方法非常巧妙，代码简洁，但不属于回溯范畴。

3. 代码实现 (Python)

```
from typing import List

class Solution:
    # 方法二：回溯法（组合视角）
    def subsets(self, nums: List[int]) -> List[List[int]]:
        """
        使用回溯算法（组合视角）生成给定数组的所有子集。
        Args:
            nums: 元素互不相同的整数数组。
        Returns:
            包含 nums 所有子集的列表。
        """
        result = [] # 存储所有子集的结果列表
        path = []   # 存储当前构建的子集路径

        def backtrack(start_index: int):
            """
            回溯函数核心逻辑。
            Args:
                start_index: 本轮选择的起始索引。
            """
            # 1. 收集结果：将当前路径的副本加入结果列表
            # 每个递归节点代表一个子集
            result.append(path[:])

            # 2. 遍历选择列表（从 start_index 开始）
            for i in range(start_index, len(nums)):
                # --- 做选择 ---
                path.append(nums[i]) # 将 nums[i] 加入当前子集
```

```

        # --- 递归进入下一层决策 ---
        # 下一次选择从  $i + 1$  开始，避免重复选择和产生重复组合
        backtrack(i + 1)

        # --- 撤销选择（回溯） ---
        path.pop() # 将 nums[i] 从路径末尾移除，恢复状态

    # 从索引 0 开始调用回溯函数
    backtrack(0)
    return result

# --- 方法三：迭代法（位运算） ---
def subsets_iterative(self, nums: List[int]) -> List[List[int]]:
    """
    使用迭代法和位运算生成所有子集。
    """
    n = len(nums)
    result = []
    # 总共有  $2^n$  个子集
    for i in range(1 << n): # 等价于 range(2**n)
        subset = []
        # 检查  $i$  的每一位二进制位
        for j in range(n):
            # 如果第  $j$  位是 1，则将 nums[j] 加入子集
            if (i >> j) & 1:
                subset.append(nums[j])
        result.append(subset)
    return result

# 示例调用
# sol = Solution()
# print(sol.subsets([1, 2, 3]))
# print(sol.subsets_iterative([1, 2, 3]))

```

4. 复杂度分析

设数组 `nums` 的长度为 N 。

- 回溯法 (组合视角):
 - 时间复杂度: $O(N * 2^N)$
 - * 状态树共有 2^N 个节点（每个节点代表一个子集）。
 - * 在每个节点处，我们需要 $O(N)$ 的时间将当前路径复制到结果列表中（最坏情况，子集可以长达 N ）。
 - * 内部的 `for` 循环的总迭代次数可以通过分摊分析，或者考虑生成所有子集元素总数的角度来看，也是 $O(N * 2^N)$ 。
 - 空间复杂度: $O(N)$

- * 递归调用栈的深度最多为 N 。
- * `path` 列表需要 $O(N)$ 的空间。
- * 结果列表 `result` 需要存储 2^N 个子集，总空间为 $O(N * 2^N)$ ，但这通常不计入算法本身的额外空间复杂度。
- 迭代法 (位运算):
 - 时间复杂度: $O(N * 2^N)$
 - * 外层循环迭代 2^N 次。
 - * 内层循环迭代 N 次，进行位运算和可能的列表添加操作。
 - 空间复杂度: $O(N)$
 - * `subset` 列表最大长度为 N 。
 - * 结果列表 `result` 空间为 $O(N * 2^N)$ (若计入)。

5. 如何在面试中讲解

1. 问题定义:
 - “目标是找出给定数组（无重复元素）的所有子集，也称为幂集。”
2. 核心思路: 回溯 (组合视角):
 - “我们可以把找子集看作是决定数组中的每个元素是否要放入子集的过程。”
 - “使用回溯算法，可以系统地构建所有可能的子集。”
 - “定义一个回溯函数 `backtrack(startIndex, currentSubset)`。”
3. 回溯过程 (组合视角):
 - “收集结果: 在递归函数的入口，就把 `currentSubset` 的副本加入最终结果列表。因为从根节点到任何节点的路径都代表一个合法的子集。”
 - “选择列表: 对于当前递归层级，我们可以选择从 `startIndex` 到数组末尾的任何一个元素加入 `currentSubset`。”
 - “递归与剪枝: 遍历从 `startIndex` 开始的元素 `nums[i]`。选择 `nums[i]` 加入 `currentSubset`，然后递归调用 `backtrack(i + 1, currentSubset)`。注意这里的 `i + 1` 是关键，它保证了我们不会选择之前的元素，从而避免了重复的组合（比如 `[1, 2]` 和 `[2, 1]` 在子集中是同一个）。”
 - “撤销选择: 递归调用返回后，需要将 `nums[i]` 从 `currentSubset` 中移除，以便尝试包含其他元素的分支。”
 - “初始调用 `backtrack(0, [])`。”
4. 复杂度分析:
 - “时间复杂度是 $O(N * 2^N)$ 。有 2^N 个子集，每个子集最多包含 N 个元素，复制子集需要时间。”
 - “空间复杂度是 $O(N)$ ，主要是递归深度和存储当前路径的列表。”
5. (可选) 迭代法 (位运算):
 - “还有一个非常巧妙的迭代方法。 N 个元素有 2^N 个子集。我们可以用一个 N 位的二进制数来表示一个子集，第 i 位为 1 表示包含 `nums[i]`，为 0 表示不包含。”
 - “遍历从 0 到 $2^N - 1$ 的所有整数。”
 - “对于每个整数，检查其二进制表示的每一位，根据位是否为 1 来决定是否将对应的 `nums` 元素加入当前子集。”
 - “这种方法时间复杂度也是 $O(N * 2^N)$ ，空间复杂度 $O(N)$ (不计结果

集)。”